

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

단원 3 : 딥러닝

DAY5. 딥러닝 모델 훈련 및 평가

목 차

1. 딥러닝 모델 훈련 및 평가		3
2. 기울기와 미분, 경사하강법		9
3. TensorFlow 소개 및 코딩 기초		17
4. CNN 모델 분석		21

1. 딥러닝 모델 훈련 및 평가

딥러닝 모델 훈련(Training)과 평가는 인공지능이 데이터를 학습하여 문제를 해결할 수 있도록 만드는 핵심 과정이다. 훈련 과정에서는 모델이 입력 데이터와 정답 데이터를 이용하여 패턴을 학습하고, 평가 과정에서는 학습된 모델이 새로운 데이터에 대해 얼마나 정확하게 예측하는지를 확인한다.

1. 딥러닝 모델 훈련(Training)

딥러닝 모델 훈련이란 신경망이 데이터로부터 규칙과 특징을 스스로 학습하는 과정이다. 예를 들어 고양이와 강아지 사진을 구분하는 모델을 만든다면, 수많은 이미지와 정답 정보를 모델에 제공하여 각 동물의 특징을 학습하도록 한다.

훈련 과정은 일반적으로 다음과 같은 단계로 이루어진다.

데이터 입력

먼저 학습에 사용할 데이터를 모델에 입력한다. 입력 데이터는 이미지, 텍스트, 음성, 센서 데이터 등 다양한 형태가 될 수 있다.

예를 들어 손글씨 숫자 인식 문제에서는 28×28 크기의 픽셀 데이터가 입력값으로 사용된다.

순전파(Forward Propagation)

입력 데이터는 신경망의 각 층을 차례대로 통과한다.

각 뉴런은 다음과 같은 계산을 수행한다.

$$z = w^T x + b$$

여기서

- x : 입력값
- w : 가중치
- b : 편향
- z : 계산 결과

계산된 결과는 활성화 함수를 통과하여 다음 층으로 전달된다.

대표적인 활성화 함수는 다음과 같다.

- Sigmoid
- Tanh
- ReLU
- Leaky ReLU
- Softmax

예측값 생성

모든 층을 통과한 후 모델은 최종 출력값을 생성한다.

예를 들어 숫자 분류 모델이라면 다음과 같은 결과가 나올 수 있다.

숫자	확률
0	0.01
1	0.02
2	0.03
3	0.91
...	...

이 경우 모델은 숫자 3이라고 예측한다.

손실 계산(Loss Calculation)

예측 결과와 실제 정답을 비교하여 오차를 계산한다.

이 오차를 손실 함수(Loss Function)라고 한다.

대표적인 손실 함수는 다음과 같다.

회귀 문제

평균제곱오차(MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

실제 값과 예측 값의 차이를 제곱하여 평균을 계산한다.

분류 문제

교차 엔트로피(Cross Entropy)

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

예측 확률과 실제 정답 확률의 차이를 측정한다.

손실값이 작을수록 모델의 성능이 좋다는 의미이다.

역전파(Backpropagation)

손실이 계산되면 모델은 오차가 발생한 원인을 찾기 위해 역전파를 수행한다.

역전파는 출력층에서 입력층 방향으로 이동하면서 각 가중치가 손실에 얼마나 영향을 주었는지 계산하는 과정이다.

이때 미분과 연쇄법칙(Chain Rule)이 사용된다.

역전파의 목적은 손실을 최소화할 수 있는 방향을 찾는 것이다.

가중치 갱신

역전파를 통해 계산된 기울기(Gradient)를 사용하여 가중치를 수정한다.

대표적인 경사하강법(Gradient Descent)의 갱신 공식은 다음과 같다.

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

여기서

- w : 가중치
- η : 학습률(Learning Rate)
- L : 손실 함수

학습률이 너무 크면 학습이 불안정해지고, 너무 작으면 학습 속도가 매우 느려진다.

Epoch와 Batch

훈련 과정은 여러 번 반복된다.

Epoch

전체 학습 데이터를 한 번 모두 사용하는 것을 1 Epoch라고 한다.

예를 들어 데이터가 10,000개이고 이를 모두 사용하면 1 Epoch가 완료된다.

Batch

데이터를 한 번에 처리하기 어려우므로 작은 묶음으로 나누어 학습한다.

예를 들어

- 데이터 개수 : 10,000개
- Batch Size : 100

이라면

100번의 반복 후 1 Epoch가 완료된다.

최적화 알고리즘

가중치를 업데이트하는 방법을 최적화 알고리즘이라고 한다.

대표적인 최적화 알고리즘은 다음과 같다.

- SGD (Stochastic Gradient Descent)
- Momentum
- AdaGrad
- RMSProp
- Adam
- AdamW

현재 실무에서는 Adam과 AdamW가 가장 널리 사용된다.

2. 딥러닝 모델 평가(Evaluation)

모델이 학습을 완료한 후에는 실제 성능을 측정해야 한다.

평가 과정은 학습에 사용되지 않은 데이터를 이용하여 수행한다.

데이터 분할

일반적으로 데이터를 다음과 같이 분할한다.

데이터 종류	용도
Train Set	모델 학습
Validation Set	하이퍼파라미터 조정
Test Set	최종 성능 평가

예를 들어

- Train : 70%
- Validation : 15%
- Test : 15%

로 분할할 수 있다.

분류 모델 평가 지표

정확도(Accuracy)

전체 예측 중 정답의 비율이다.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

쉽게 이해할 수 있지만 데이터 불균형이 심한 경우 신뢰하기 어렵다.

정밀도(Precision)

양성이라고 예측한 것 중 실제 양성의 비율이다.

$$Precision = \frac{TP}{TP+FP}$$

재현율(Recall)

실제 양성 중 모델이 찾아낸 비율이다.

$$Recall = \frac{TP}{TP+FN}$$

F1 Score

정밀도와 재현율의 조화평균이다.

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

데이터가 불균형한 경우 자주 사용된다.

회귀 모델 평가 지표

대표적으로 다음을 사용한다.

MAE

평균절대오차

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

RMSE

평균제곱근오차

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

결정계수(R²)

모델 설명력을 나타낸다.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \text{bary})^2}$$

1에 가까울수록 좋은 모델이다.

3. 과적합과 성능 개선

훈련 성능은 높지만 새로운 데이터에서 성능이 낮은 현상을 과적합(Overfitting)이라고 한다.
과적합을 줄이기 위해 다음 기법을 사용한다.

- 데이터 증강(Data Augmentation)
- Dropout
- Batch Normalization
- Early Stopping
- 정규화(L1, L2)
- 더 많은 데이터 확보

반대로 모델이 충분히 학습하지 못한 상태는 과소적합(Underfitting)이라고 한다.

2. 기울기와 미분, 경사하강법

딥러닝 모델은 손실 함수의 기울기를 미분을 통해 계산하고, 손실이 감소하는 방향으로 가중치를 수정한다. 이러한 과정이 반복되면서 모델은 점점 더 정확한 예측을 수행하게 된다. 따라서 딥러닝의 학습 과정은 단순히 데이터를 외우는 것이 아니라, 미분과 기울기를 이용하여 최적의 가중치를 찾아가는 수학적 최적화 과정이라고 이해할 수 있다.

Loss를 minimize하는 w, b 를 구하고 싶다.

- > Random Search? 어느 세월에..
- > Loss 값을 통해서 구할 수 있을까?

답

러닝 모델을 학습시키는 가장 중요한 목표는 예측 오차(Loss)를 최소화하는 것이다. 모델 내부에는 수많은 가중치(Weight)와 편향(Bias)이 존재하는데, 어떤 값을 사용해야 가장 좋은 결과를 얻을 수 있는지 찾아야 한다.

모든 가중치를 무작위로 바꿔가며 최적의 값을 찾는 방법도 생각할 수 있지만, 딥러닝 모델은 수천 개에서 수억 개의 파라미터를 가지고 있기 때문에 현실적으로 불가능하다.

따라서 딥러닝은 현재 오차 정보를 이용하여 더 좋은 방향으로 이동하는 방법을 사용한다. 이때 핵심이 되는 개념이 바로 **기울기(Gradient)** 이다.

Loss를 minimize하는 w, b 를 구하고 싶다.

-> Random Search? 어느 세월에..

-> Loss 값을 통해서 구할 수 없을까? Gradient Descent!

딥러닝에서는 손실 함수의 값을 줄이기 위해 **경사하강법(Gradient Descent)** 을 사용한다.

경사하강법은 현재 위치에서 손실 함수의 기울기를 계산한 뒤, 오차가 줄어드는 방향으로 조금씩 이동하는 방법이다.

예를 들어 산 정상에서 가장 낮은 계곡으로 내려간다고 생각하면 된다.

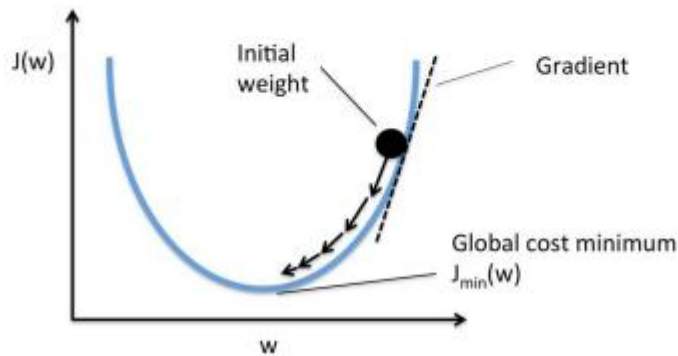
- 현재 위치 확인
- 주변 경사도 확인
- 가장 낮아지는 방향 선택
- 조금 이동
- 다시 경사도 계산

이 과정을 반복하면서 점점 더 좋은 가중치를 찾는다.

Loss를 minimize하는 w, b 를 구하고 싶다.

-> Random Search? 어느 세월에..

-> Loss 값을 통해서 구할 수 있을까? Gradient Descent!



그래프의

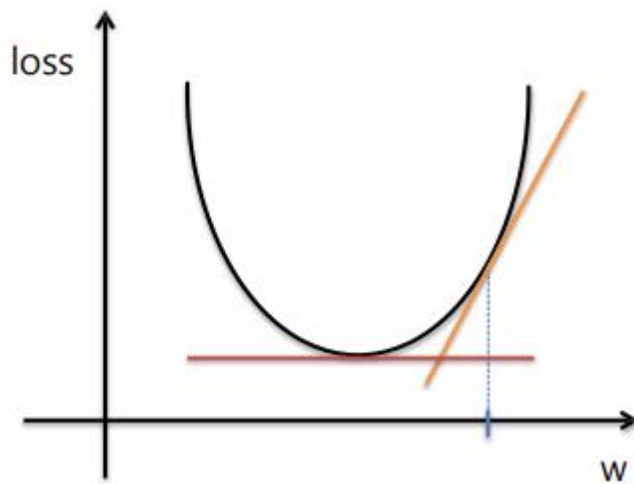
곡선은 손실 함수(Loss Function)를 의미한다.

현재 가중치가 최적값이 아니라면 손실 함수 위의 어느 한 지점에 위치하게 된다.

기울기(Gradient)는 현재 위치에서 그래프가 얼마나 가파르게 증가하거나 감소하는지를 나타낸다.

- 기울기가 양수이면 왼쪽 방향 이동
- 기울기가 음수이면 오른쪽 방향 이동
- 기울기가 0 이면 최적점 근처

딥러닝은 이러한 기울기 정보를 이용하여 손실이 가장 작은 지점을 찾아간다.



그냥 바로 loss를 w 에 대해 미분했을 때 그 값이 0이 되는 w 를 찾으면 안될까?

손실

함수의 기울기가 0 이 되는 지점을 직접 찾으면 최적의 가중치를 얻을 수 있을 것처럼 보인다.

수학적으로 함수의 최솟값은 다음과 같이 표현된다.

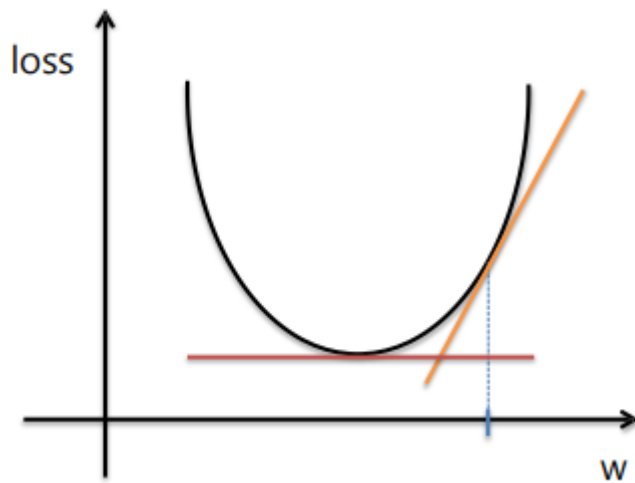
$$\frac{dL}{dw} = 0$$

여기서

- L : 손실 함수
- w : 가중치

즉, 손실 함수를 가중치에 대해 미분했을 때 0 이 되는 지점을 찾으면 된다.

그러나 딥러닝에서는 문제가 훨씬 복잡해진다. 수많은 가중치가 동시에 존재하기 때문이다.



그냥 바로 loss를 w 에 대해 미분했을 때 그 값이 0이 되는 w 를 찾으면 안될까?

loss를 minimize하는 w 를 구하려면 $w = (x^T x)^{-1} x^T y$ 식을 풀어야 하는데, 변수가 많아지고 matrix가 커지면 복잡도가 $O(n^3)$ 이기 때문에 exponential 하게 증가한다. 변수가 많아질수록 계산이 거의 불가능.

실제 딥러닝 모델은 수백만 개 이상의 가중치를 가진다.

만약 모든 가중치에 대한 최적해를 수학적으로 한 번에 계산하려고 하면 엄청난 양의 행렬 연산이 필요하다.

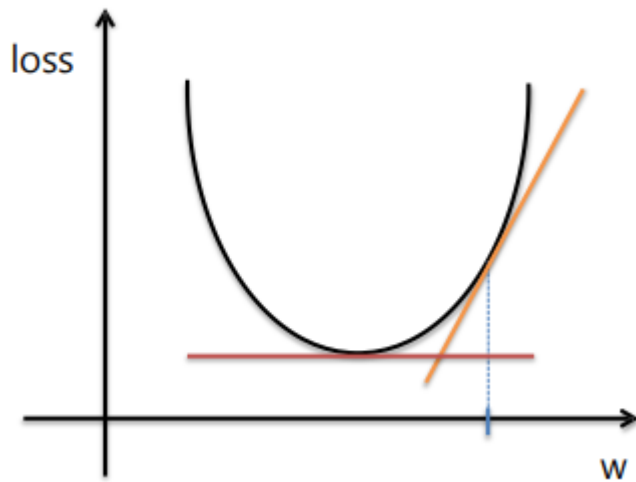
예를 들어

- 이미지 분류 CNN
- 자연어 처리 Transformer
- GPT와 같은 대규모 언어모델

은 수억~수천억 개의 파라미터를 가진다.

이러한 모델에서는 정확한 해를 직접 구하는 것이 사실상 불가능하다.

그래서 딥러닝은 최적해를 한 번에 구하는 대신 반복적으로 개선하는 방식을 사용한다.



바로 minimum이 되는 지점을 구하기보다, 현재 loss에 대한 w 의 gradient(경사도)를 구하여 gradient x learning rate만큼 w 를 업데이트

딥러닝은 현재 손실 함수의 기울기를 계산한 뒤 그 반대 방향으로 이동한다.

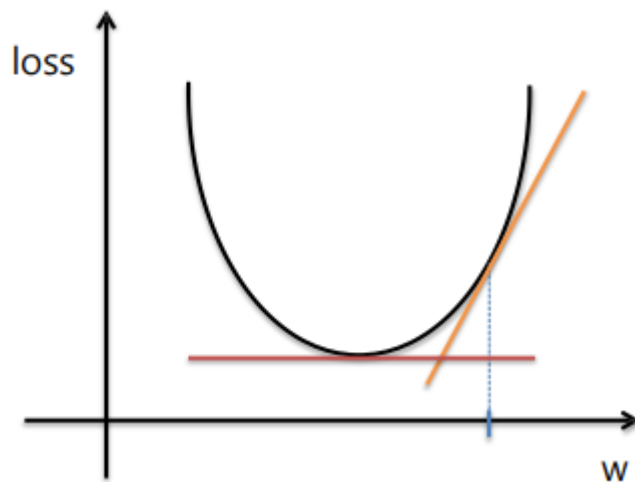
기울기는 다음과 같이 정의된다.

$$\text{Gradient} = \frac{\partial L}{\partial w}$$

기울기의 의미는 다음과 같다.

- 기울기 값이 크다 → 손실이 급격히 변함
- 기울기 값이 작다 → 손실 변화가 적음
- 기울기 값이 0에 가까움 → 최적점에 가까움

딥러닝은 매 학습 단계마다 이 값을 계산하여 가중치를 수정한다.



바로 minimum이 되는 지점을 구하기보다, 현재 loss에 대한 w 의 **gradient(경사도)**를 구하여 **gradient x learning rate**만큼 w 를 업데이트

$$w_{t+1} = w_t - \text{gradient} * \text{learning_rate}$$

기울기를 계산한 후에는 가중치를 업데이트한다.

업데이트 공식은 다음과 같다.

$$w_{t+1} = w_t - \eta \frac{\partial L}{\partial w}$$

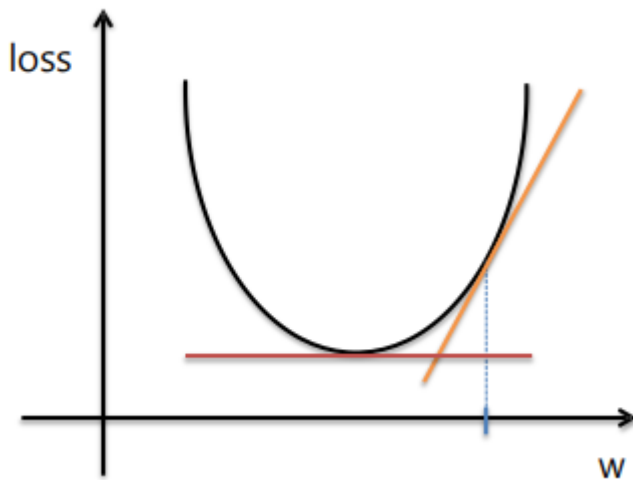
여기서

- w_t : 현재 가중치
- w_{t+1} : 다음 가중치
- η : 학습률(Learning Rate)
- $\frac{\partial L}{\partial w}$: 기울기

학습률은 이동 속도를 결정한다.

학습률이 너무 크면 최적점을 지나쳐 버리고, 너무 작으면 학습 시간이 매우 오래 걸린다.

따라서 적절한 학습률을 설정하는 것이 매우 중요하다.



바로 minimum이 되는 지점을 구하기보다, 현재 loss에 대한 w 의 **gradient(경사도)**를 구하여 $\text{gradient} \times \text{learning_rate}$ 만큼 w 를 업데이트

$$w_{t+1} = w_t - \text{gradient} * \text{learning_rate}$$

계속 업데이트 하다 보면 optimal한 w 에 근접하게 된다.

가중치 업데이트를 반복하면 손실 함수의 값은 점점 작아진다.

이 과정을 수백 번, 수천 번, 수만 번 반복하면서 모델은 최적의 가중치에 가까워진다.

이러한 반복 학습 과정을 Epoch 라고 하며, 딥러닝 모델은 다음 과정을 계속 수행한다.

1. 입력 데이터 전달
2. 예측 수행
3. 손실 계산
4. 미분 수행
5. 기울기 계산
6. 가중치 수정
7. 다시 예측

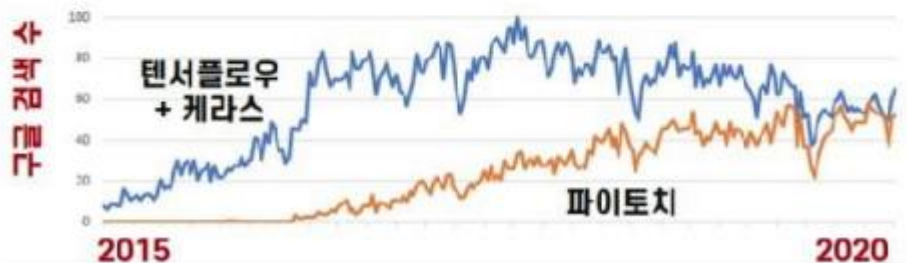
결국 딥러닝 학습은 "미분을 이용해 기울기를 계산하고, 기울기를 이용해 가중치를 반복 수정하는 과정" 이라고 볼 수 있다.

3. TensorFlow 소개 및 코딩 기초

2) 기계 학습 코딩 Package의 3대 천황

(1) 기계 학습 코딩 Package의 3대 천황

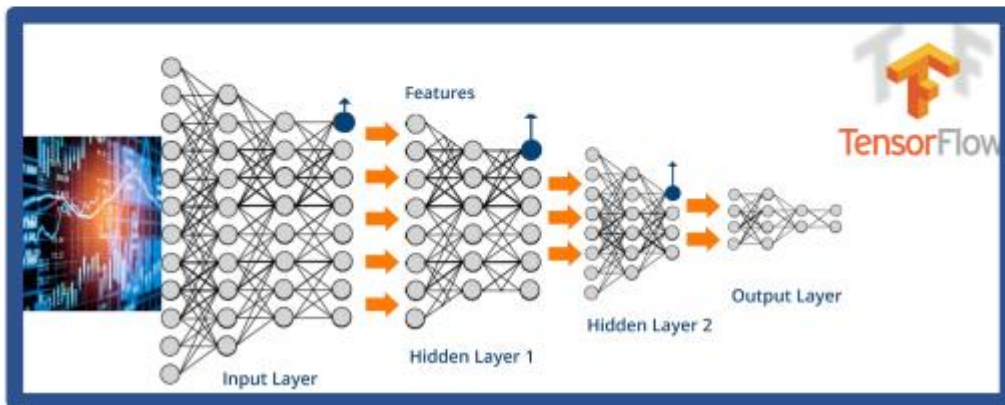
- 모두 파이썬기반



3) 텐서플로우(Tensorflow) 소개

(1) 파이썬 기계 학습용 무료 라이브러리

- 구글 브레인팀이 자체 제작
- V1.0 소스 공개 (2015)
- 구글의 TPU (tensor processing unit)과 연동
- V2.0 소스 공개 (2019): PyTorch 기술 많이 도입



4) 케라스(Keras) 소개

(1) 파이썬 기계 학습용 무료 라이브러리

- 구글의 François Chollet 팀이 공개(2015)
- 텐서플로우를 쉽게 쓰도록 하는 도우미 역할
- 인공신경망 구현, 학습, 평가가 많이 쉬워짐
- 대신 텐서플로우에 비해 느리고, 기업형 서비스는 불가



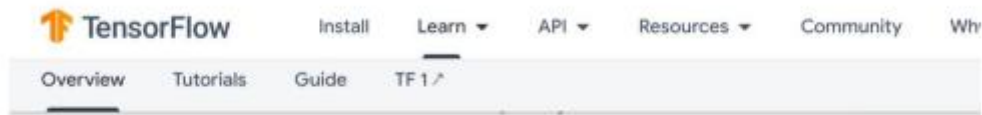
VS



2) 텐서플로를 활용한 MNIST 손글씨 인식

(1) 텐서플로를 활용한 MNIST 손글씨 인식

- www.tensorflow.org에서 제공



Keras
15줄
딥러닝



```
import tensorflow as tf
mnist = tf.keras.datasets.mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5)
model.evaluate(x_test, y_test)
```

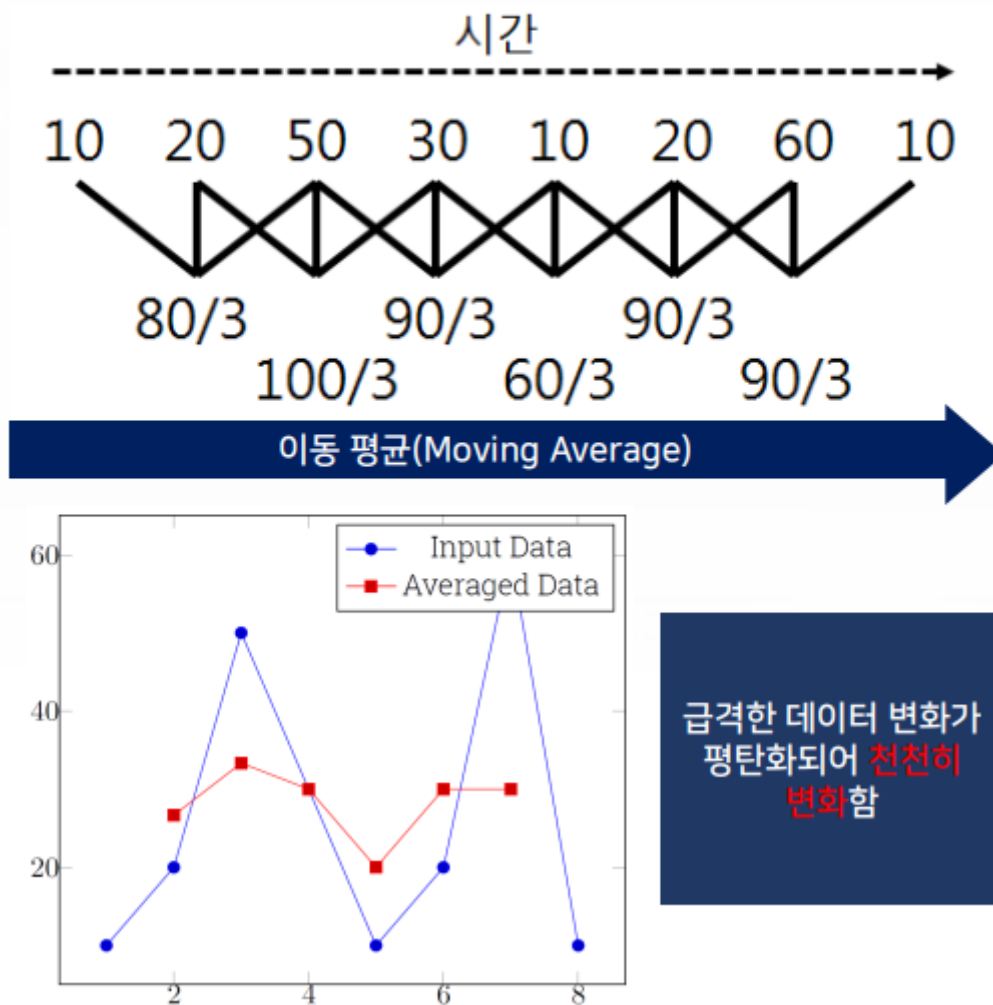
4. CNN 모델 분석

1. 합성곱 연산의 이해

1) 이동 평균(Moving Average)

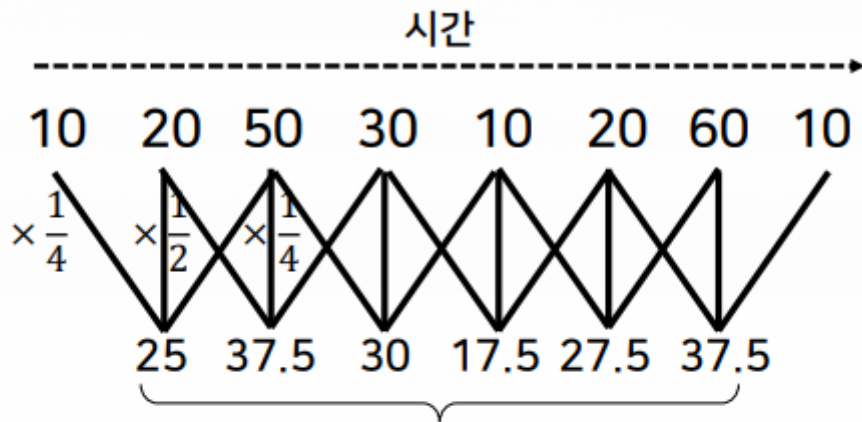
(1) 이동 평균(Moving Average)

- 데이터를 따라가며 평균 구하기

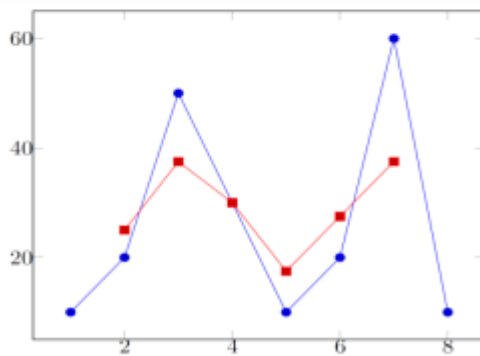


1) 이동 평균(Moving Average)

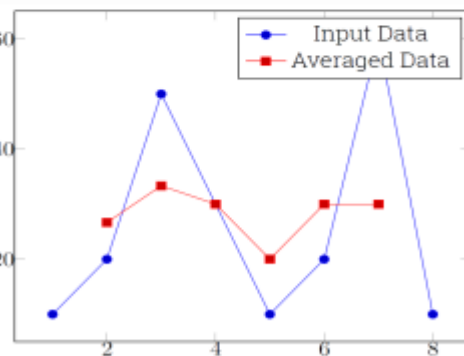
(1) 이동 평균(Moving Average)



합성곱(Convolution) 연산



이동 평균(Moving Average)



합성곱(Convolution) 연산

2) 합성곱(Convolution) 연산

(1) 합성곱(Convolution) 연산

- 커널(Kernel)

- 곱해지는 수를 다 모은 것

Ex) $h = \begin{bmatrix} \frac{1}{4} & \frac{1}{2} & \frac{1}{4} \end{bmatrix}$ 에 대한 합성곱 연산

- 계수(Weight)

- 커널에 있는 값들

커널 $h = \begin{bmatrix} \frac{1}{4} & \frac{1}{2} & \frac{1}{4} \end{bmatrix}$ 에 대한 합성곱 연산

$$y[n] = h[-1] \times x[n-1] + h[0] \times x[n] + h[1] \times x[n+1]$$

일반적으로 커널 $h \left[-\frac{K-1}{2} \right] \sim h \left[\frac{K-1}{2} \right]$ 에 대해

$$y[n] = \sum_{k=-\frac{K-1}{2}}^{\frac{K-1}{2}} h[k] \times x[n+k]$$

3) 합성곱 활용

(1) 필터링(Filtering)

- 합성곱 연산은 신호 처리에서 매우 많이 이용됨
- 주로 많이 이용되는 곳이 필터링
- 입력 신호에 대해 입력데이터에 대해서 내가 원하는 주파수만 걸러내는 것

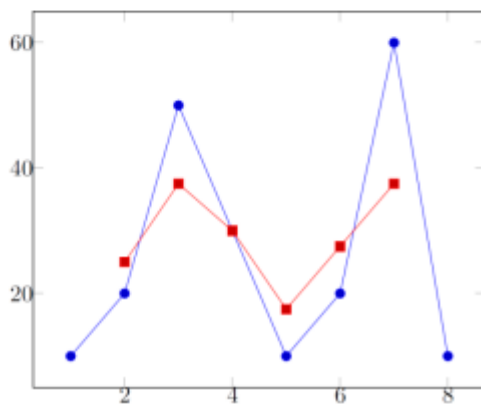
(2) 저역 통과 필터(Low Pass Filter, LPF)

낮은 주파수 성분 → 통과

시간에 따라서 값이 천천히 바뀌는 것

높은 주파수 성분 → 제거

시간에 따라서 값이 급격하게 변하는 것



높은 주파수가 많음

저역 통과 필터

낮은 주파수만 남음

3) 합성곱 활용

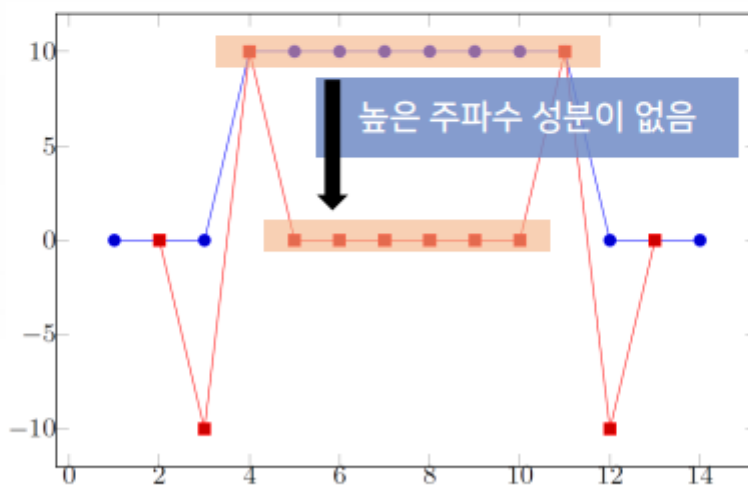
(3) 고역 통과 필터(High Pass Filter, HPF)

낮은 주파수 성분 → 제거

높은 주파수 성분 → 통과

▪ 예

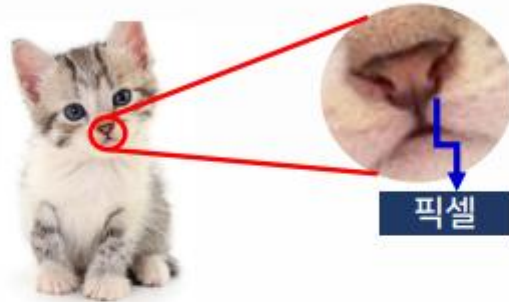
$$h = [-1 \quad 2 \quad -1]$$



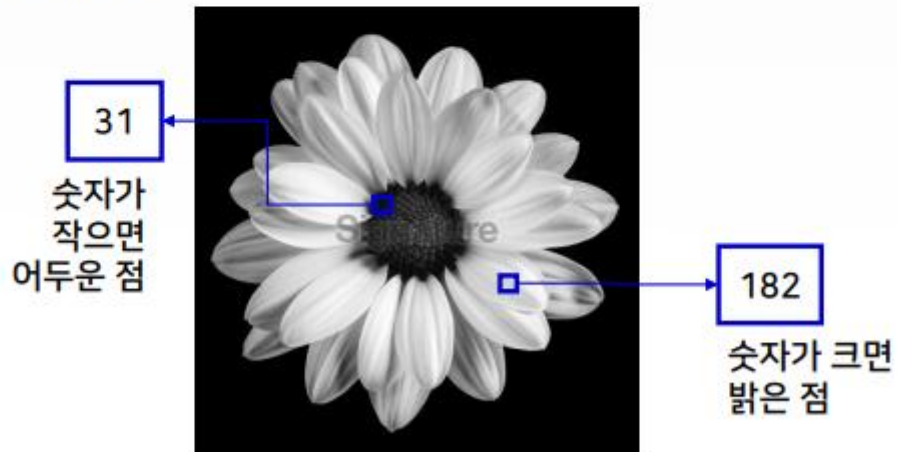
1) 영상에서의 합성곱

(1) 영상 데이터

- 사진이 있을 때 아주 작은 점들로 이루어져 있음
- 각각의 점을 픽셀이라고 부름



- 흑백 영상



2. 영상에서의 합성곱

1) 영상에서의 합성곱

(1) 영상 데이터

▪ 컬러 영상



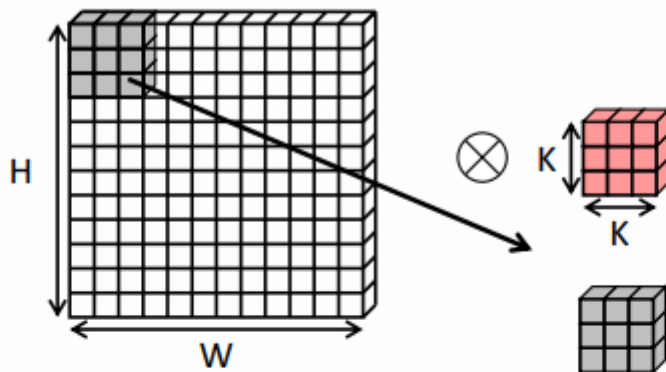
▪ 해상도(Resolution)

- 하나의 사진에 있는 픽셀의 갯수



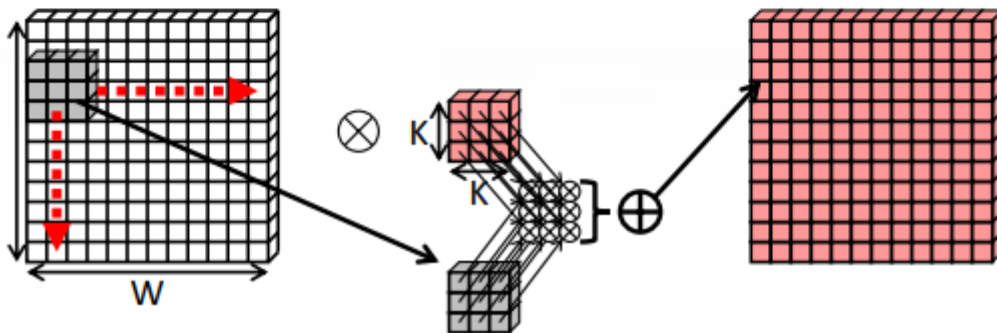
1) 영상에서의 합성곱

(2) 영상에서의 합성곱



입력 데이터
2차원

커널
2차원



2) 영상 필터

(1) 필터링(Filtering)

- 저역 통과 필터
 - 낮은 주파수 성분만 통과
- 고역 통과 필터
 - 높은 주파수 성분만 통과

(2) 저역 통과 필터의 예

- 경계 부분은 높은 주파수 성분이 많음
 - 경계 부분이 흐릿해짐

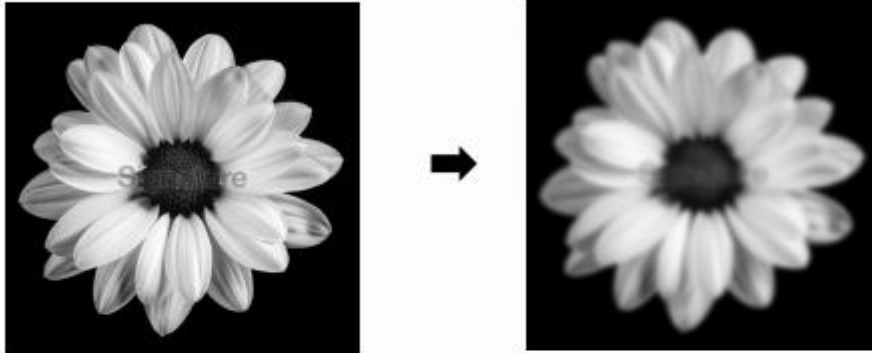
1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

1/16	1/8	1/16
1/8	1/4	1/8
1/16	1/8	1/16

2) 영상 필터

(2) 저역 통과 필터의 예

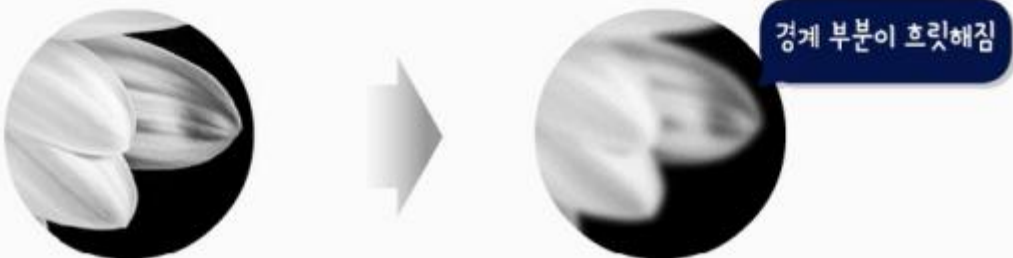
- 저역통과필터로 합성곱 연산한 결과



- 경계 부분은 높은 주파수 성분이 많음



경계 부분이 흐릿해짐



2) 영상 필터

(2) 저역 통과 필터의 예

- 포토샵의 블러링

- 저역통과필터를 이용한 컨벌루션 연산



2) 영상 필터

(3) 고역 통과 필터의 예

Prewitt Kernels

가로방향의 엣지 검출

위와 아래 사이 값이
많이 바뀌는 경계를
검출하는 커널

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

세로방향 경계 검출

왼쪽과 오른쪽 사이
값이 많이 바뀌는 경계를
검출하는 커널

Sobel Kernels

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

0	1	2
-1	0	1
-2	-1	0

-2	-1	0
-1	0	1
0	1	2



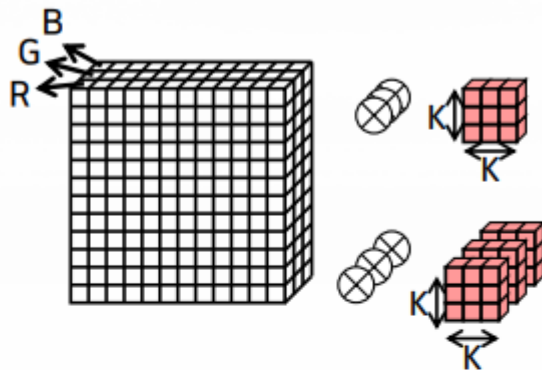
고역통과필터



- 경계 부분만 검출

3) 컬러 영상

(1) 컬러 영상에서의 필터링



컨벌루션 연산 방법

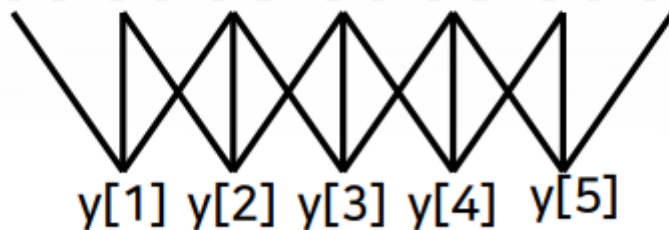
- RGB 채널에 대해 동일한 커널 적용
- RGB 각각의 채널에 서로 다른 커널 적용

(2) 데이터 개수

- 입력데이터에 비해 출력데이터가 줄어드는 현상 발생

$x[0]$ $x[1]$ $x[2]$ $x[3]$ $x[4]$ $x[5]$ $x[6]$

7개



5개

- N개의 데이터, 크기가 K인 커널

N개 데이터 $\rightarrow$ **$N-K+1$ 개** 데이터

3) 컬러 영상

(3) 해결법

- 입력 데이터 개수와 출력 데이터 개수를 똑같이 하려면?

Padding

0 x[0] x[1] x[2] x[3] x[4] x[5] x[6] 0

Copying

x[0] x[0] x[1] x[2] x[3] x[4] x[5] x[6] x[6]

Warping

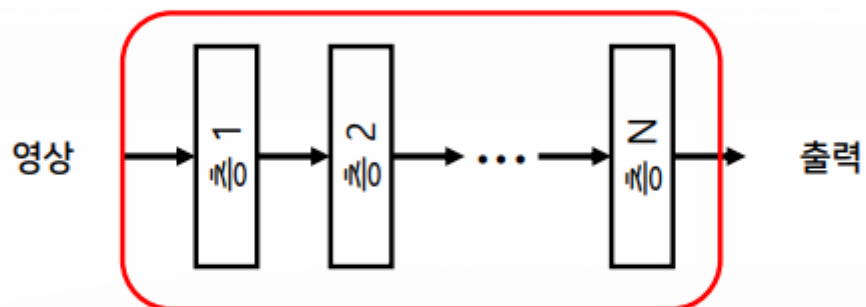
x[6] x[0] x[1] x[2] x[3] x[4] x[5] x[6] x[0]

3. CNN 기본 구조의 합성곱 층

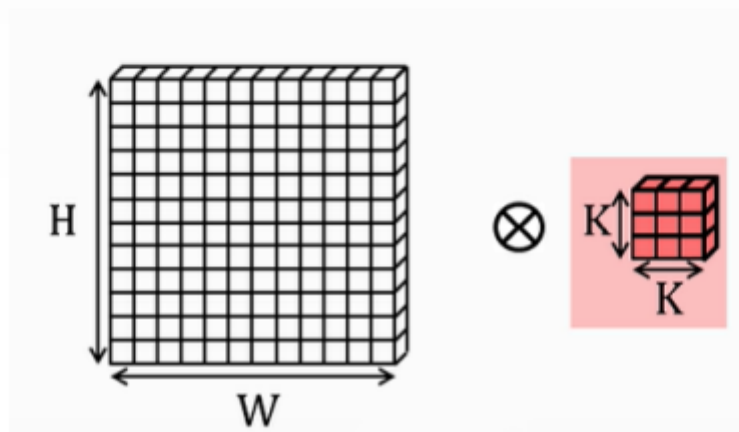
1) Convolutional Neural Network

(1) 합성곱 신경망(Convolutional Neural Network, CNN)

- 영상 처리에 주로 이용
- 합성곱 층
 - 합성곱 기반 연산

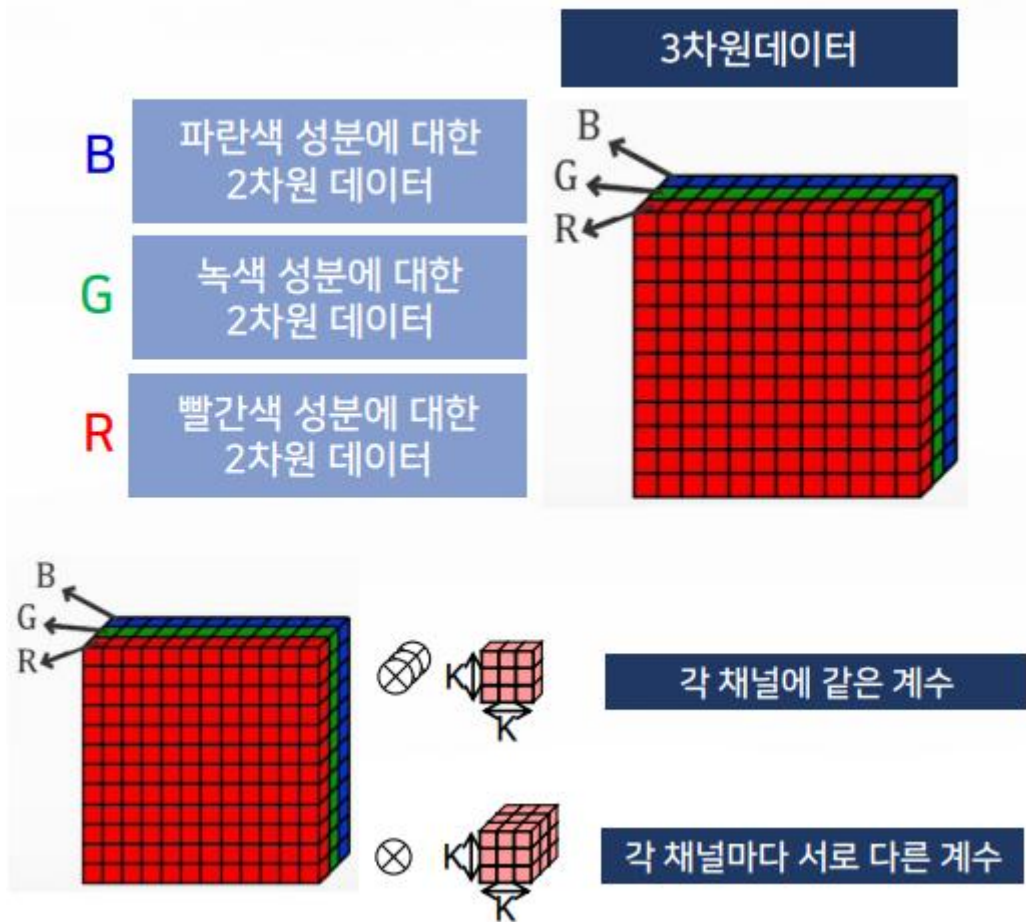


- CNN 합성곱 신경망에서의 학습



2) 첫 번째 층

(1) 첫 번째 층



2) 첫 번째 층

(1) 첫 번째 층

- CNN에서의 학습은 무엇일까?



학습을 통해
커널의 역할을 확인

Convolution을
같은 계수로 할지
서로 다른 계수들로 할지
미리 알 수 없음

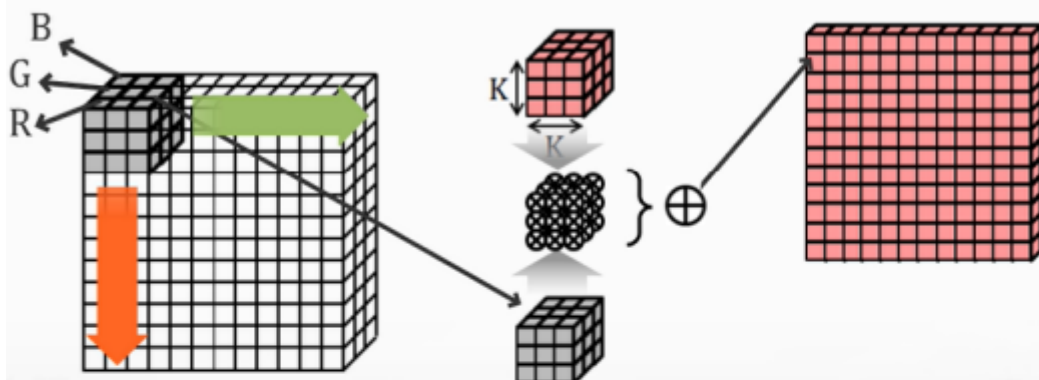
'RGB에 똑같은 계수를 한다?

'RGB에 각각 다른 계수들을 가지고
곱하는 커널'로 가정

II

2) 첫 번째 층

(1) 첫 번째 층



- 몇 개의 커널이 필요할까?

한 개의 커널



한 개의 엣지 검출

Prewitt Kernels

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

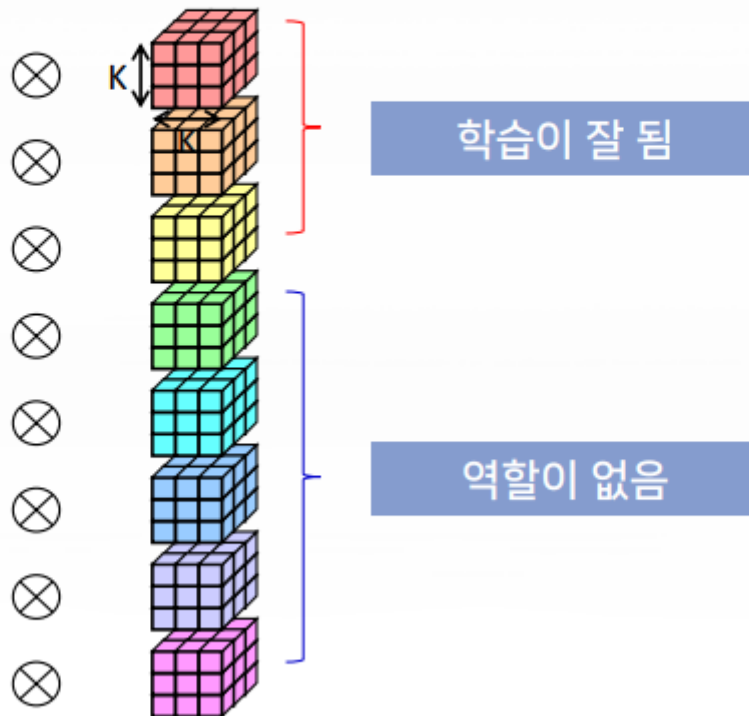


한 종류의 피쳐만 검출

2) 첫 번째 층

(1) 첫 번째 층

- 첫 번째 층에서 몇 종류의 피쳐를 검출하는게 좋을까?
 - 너무 적은 개수의 커널로는 충분한 피쳐 검출 불가능

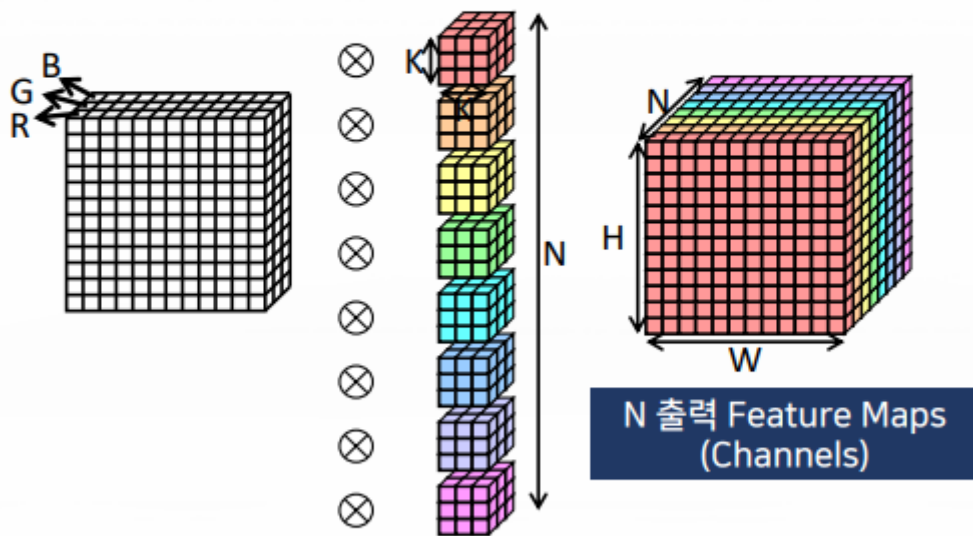


2) 첫 번째 층

(1) 첫 번째 층

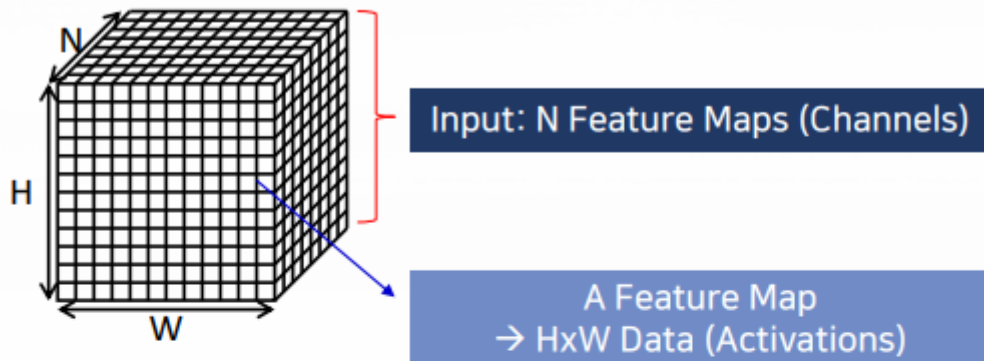
- N 출력 Feature Maps

- N개의 커널로 Convolution 했을 때 출력되는 데이터

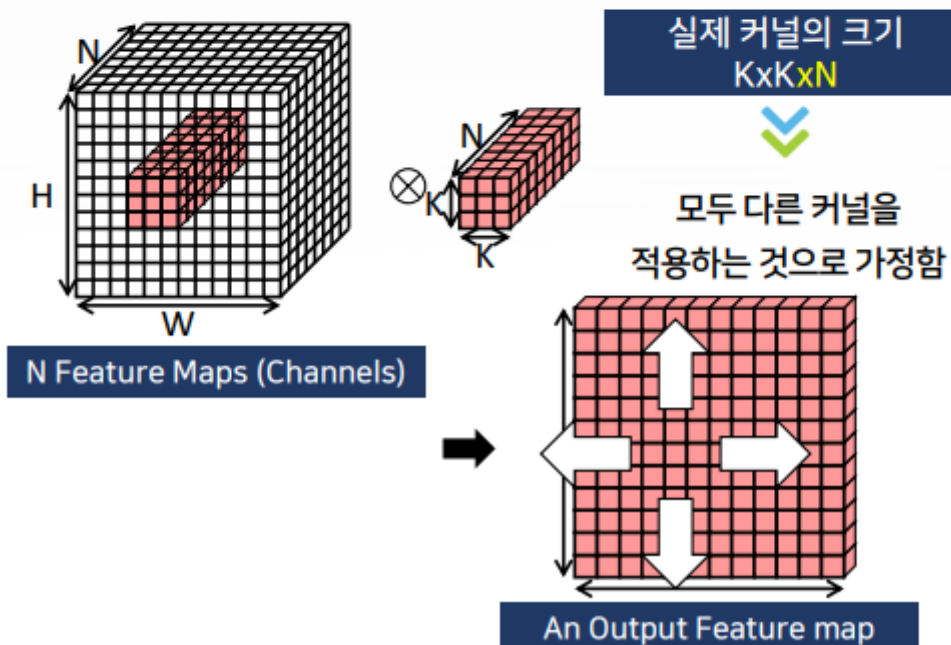


3) 다음 층

(1) 첫 번째 층에서의 출력

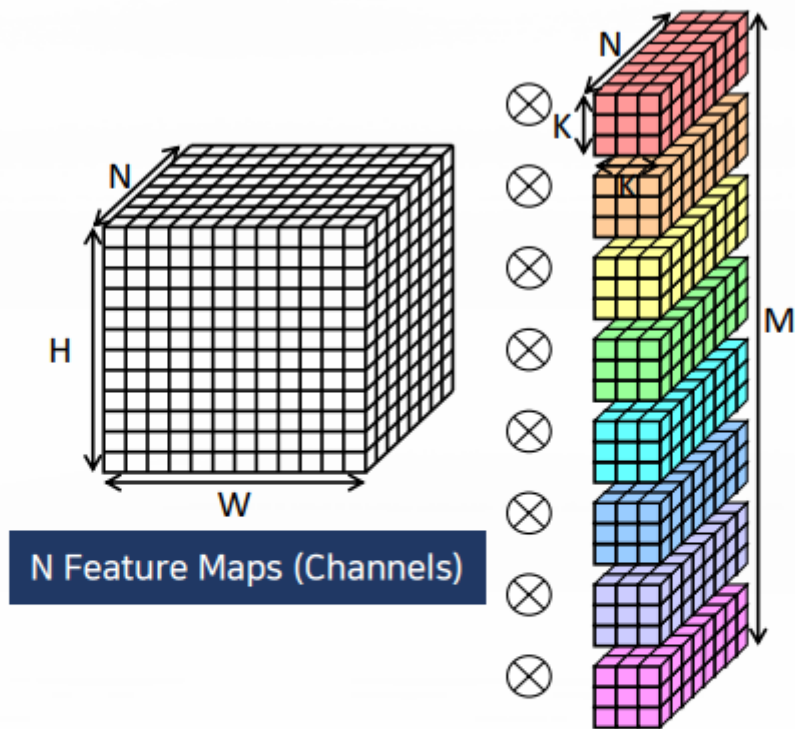


(2) 다음 층에서의 Convolution



4) 합성곱 층(Convolutional Layer)

(1) 합성곱 층(Convolutional Layer)



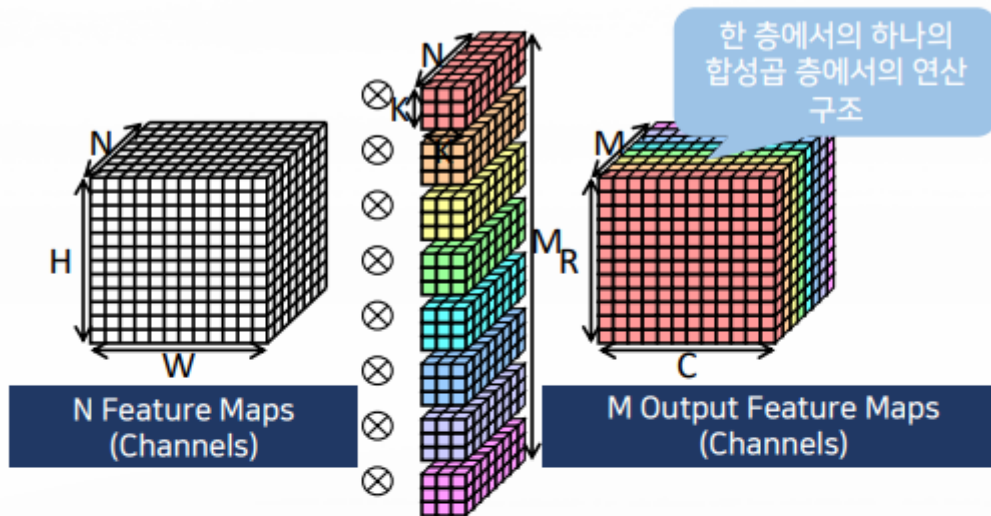
많은 피쳐

높은 자유도

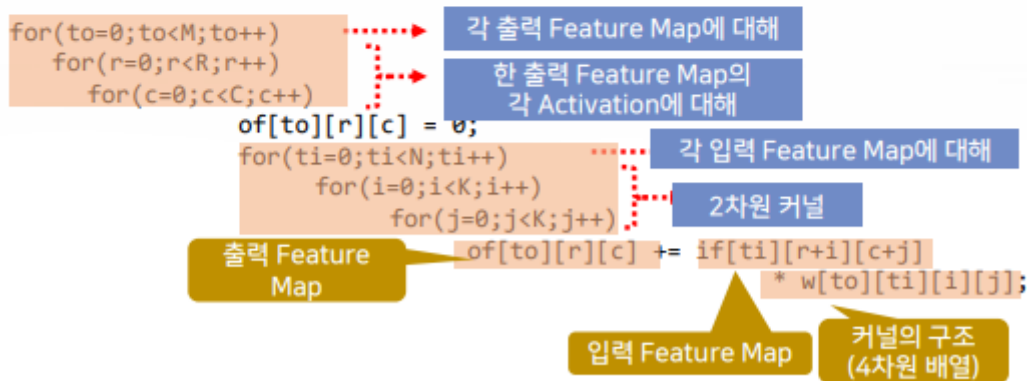
제대로 된 학습

4) 합성곱 층(Convolutional Layer)

(1) 합성곱 층(Convolutional Layer)



(2) 반복문 표현



6중 반복문

4) 합성곱 층(Convolutional Layer)

(3) 곱셈의 횟수

곱셈의 횟수: $M \times R \times C \times N \times K \times K$

$$M = N = 100$$

$$R = C = 200$$

$$K = 3$$

$$3.6 \times 10^9$$

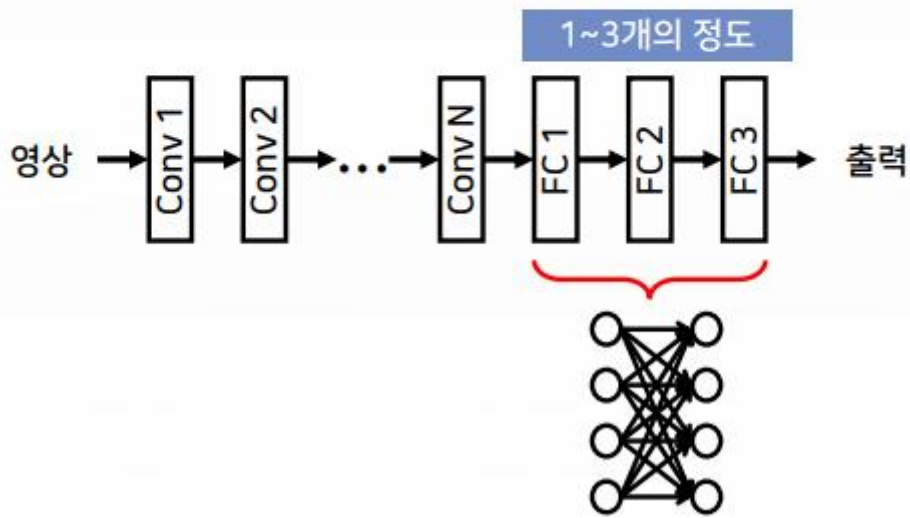
3차원적인 합성곱 연산

3x3 커널 → 3x3x(입력 채널 개수) 커널

1) 완전 연결 층

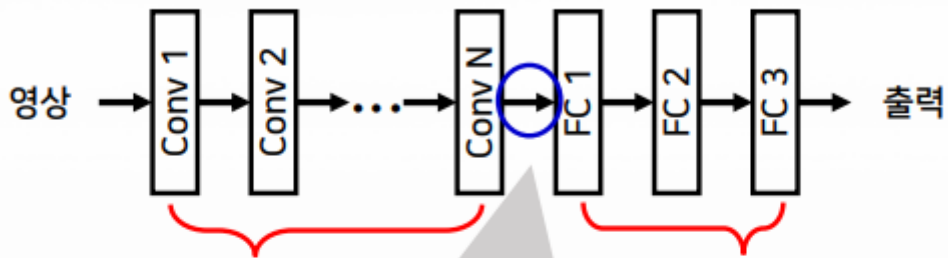
(1) 완전 연결 층(Fully-Connected Layer)

- 층의 구조
 - 일반적인 신경망층
- 입력 데이터가 있을 때, 이것들을 가지고 계수를 곱한 다음 더해서 출력을 만들어내는 형태의 층



1) 완전 연결 층

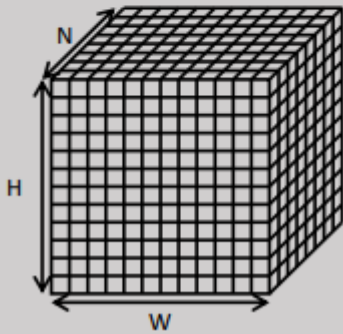
(1) 완전 연결 층(Fully-Connected Layer)



출력 데이터 : 3차원적인 구조

입력 데이터 : 1차원적인 구조

3D Feature Map



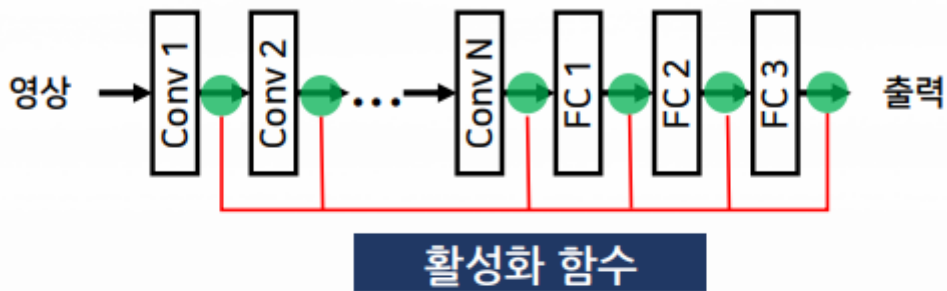
1D vector



논리적으로 구조
변환 후
완전연결층으로
처리

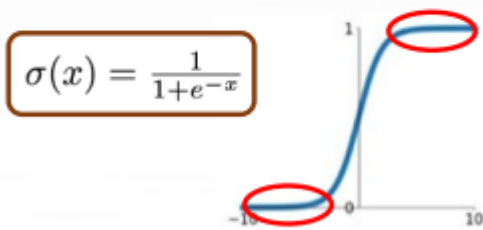
2) 활성화 함수

(1) 활성화 함수

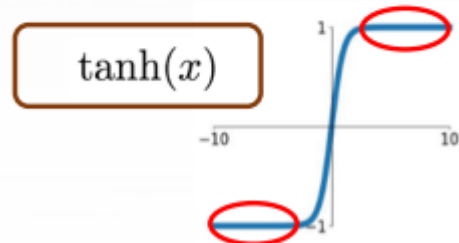


(2) 이전의 활성화 함수

Sigmoid



Tanh



미분값이 매우 작은 영역 → 학습이 잘 안 됨

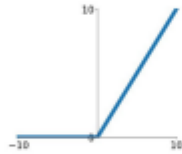
2) 활성화 함수

(3) 현재의 활성화 함수

ReLU (Rectified Linear Unit)

Leaky ReLU

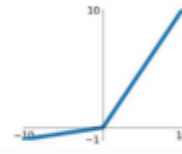
$$\max(0, x)$$



음수 입력 → 0 출력

양수 입력 → 동일값 출력

$$\max(0.1x, x)$$



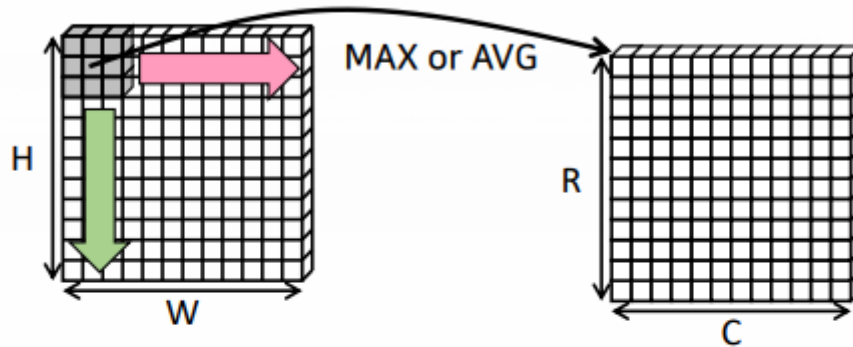
양수 입력 → 동일값 출력

음 입력
→ 작은 값을 곱하여 출력

3) Pooling 층

(1) Pooling 층

- 각 입력 Feature Map마다 따로 연산 적용



→ 원래는 3차원이나 각 2차원 데이터에 대해 따로 함

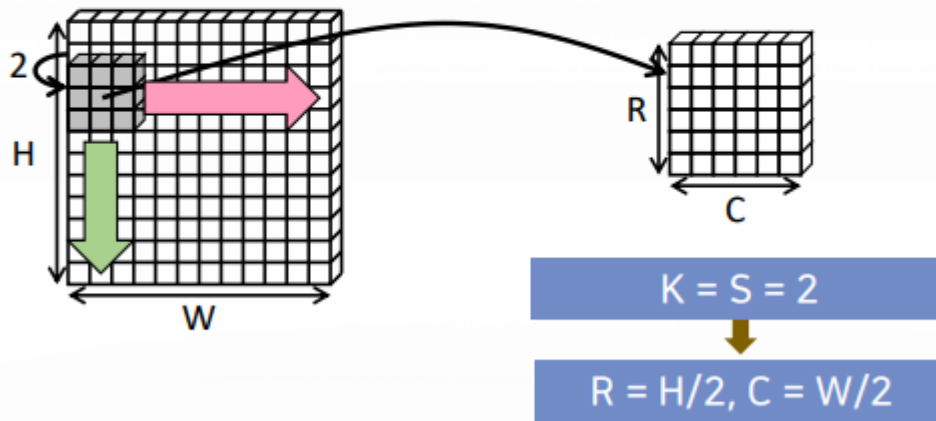
(2) 합성곱 연산과 Pooling Layer의 차이점

- 합성곱 연산
 - 채널 방향으로 3차원 데이터를 다 곱해서 더하는 것
- Pooling Layer
 - 각 Feature Map에 연산을 따로 수행함
 - 2차원적인 연산을 각 채널마다 따로 하여 3차원적인 데이터를 얻어냄
 - MAX 또는 AVG 연산 사용

3) Pooling 층

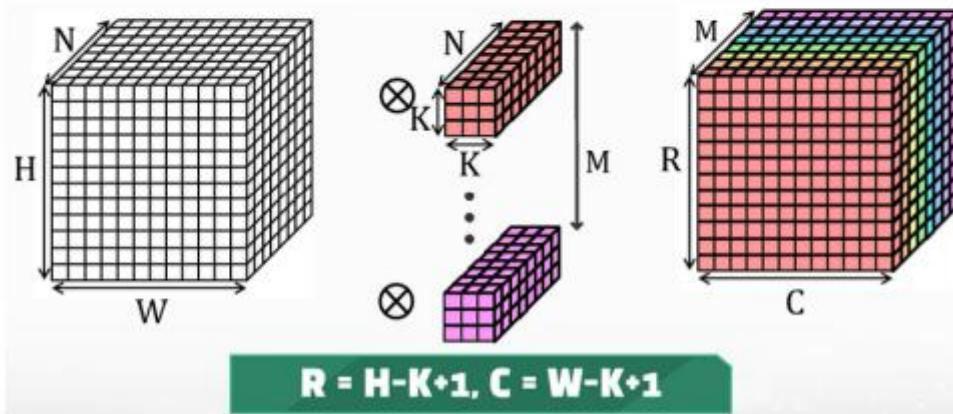
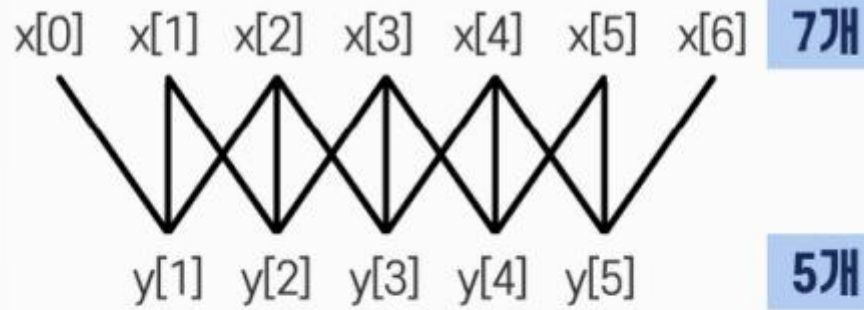
(3) Stride

With Stride = 2 → 2칸씩 이동



4) Padding을 넣은 합성곱

(1) 합성곱

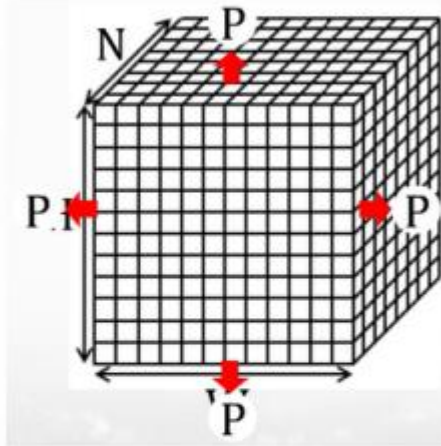


- 3x3 커널의 경우 가로 세로 크기가 2씩 감소

4) Padding을 넣은 합성곱

(1) 합성곱

- CNN에서는 Padding을 많이 사용함



Padding with P

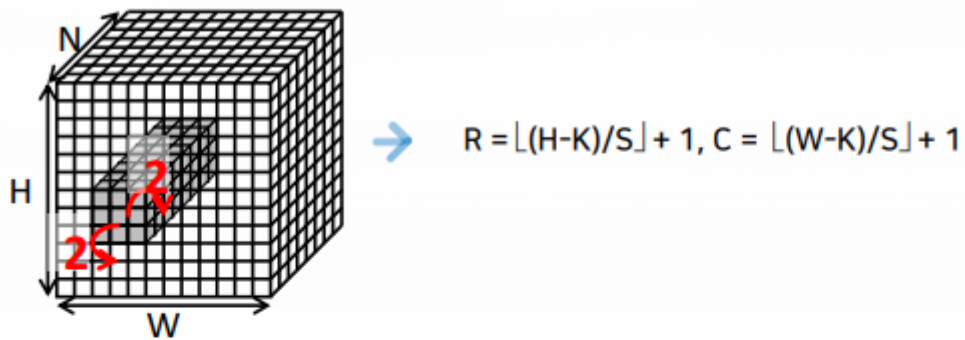
$$\rightarrow R = H + 2P - K + 1, C = W + 2P - K + 1$$

$$P = \frac{K-1}{2}$$

$$\rightarrow R = H \text{ and } C = W$$

5) Stride를 넣은 합성곱

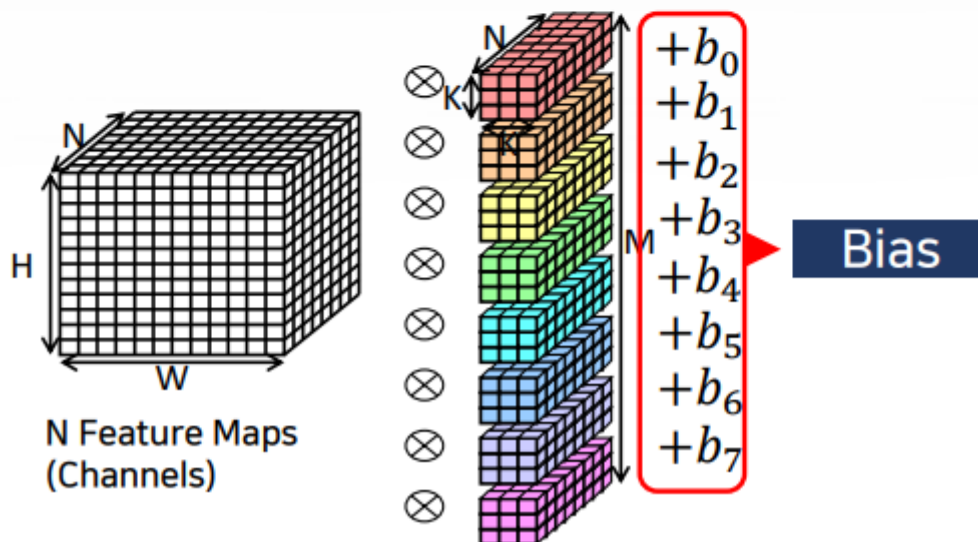
(1) Stride를 넣은 합성곱



6) 합성곱 층의 Bias

(1) 합성곱 층의 Bias

- 어느 한 출력 Feature Map의 Activation들에는 동일한 Bias를 더함



7) 배치 정규화(Batch Normalization)

(1) 공변량 쉬프트(Covariate Shift)



- 학습 : 각 층의 계수를 바꾸는 것
- 학습이 진행되면 각 층에서 계수가 바뀜
 - 같은 입력을 넣어도 층의 출력이 바뀜
- 각 층의 입력 데이터의 분포가 계속해서 바뀌게 됨
- 층이 더 깊어질수록 더 많이 바뀌게 됨

(2) 정규화(Normalization)

정규화(Normalization)

어떤 층 출력데이터 전체에 대해서

- μ : x_i 의 평균
- σ : x_i 의 표준 편차

$$\hat{x}_i = \frac{x_i - \mu}{\sigma}$$

평균은 0으로 표준편차는 1로 맞춰짐

다음 층에서는 입력 데이터의 분포가 동일해짐

7) 배치 정규화(Batch Normalization)

(3) 정규화의 문제점

- 평균과 표준편차를 구하기가 어려움

(4) 미니 배치(Mini-Batch)

- 확률적 경사하강법(Stochastic Gradient Descent)에서 미니 배치 단위로 계수를 업데이트
 - 미니 배치 단위마다 평균과 표준편차를 계산함
 - 배치 정규화

(5) 배치 정규화 층(Batch Normalization Layer)

각 Feature Map에 대해

- μ_B : 미니 배치에서 x_i 의 평균
 - σ_B : 미니 배치에서 x_i 의 표준 편차
- $$\hat{x}_i = \frac{x_i - \mu_B}{\sigma_B}$$

활성화 함수 이전에 배치 정규화 층을 넣음

7) 배치 정규화(Batch Normalization)

(5) 배치 정규화 층(Batch Normalization Layer)

- 평균=0이고 표준 편차=1인 분포가 정말 좋을까?
 - 스케일 층(Scale Layer)에서 스케일(γ)과 쉬프트(β)를 학습
 - 보통 배치정규화 층은 스케일 층과 같이 붙어서 다니는 것이 일반적

$$y_i = \gamma \hat{x}_i + \beta$$

8) 배치 정규화의 효과

(1) 배치 정규화의 효과

학습률(Learning Rate)을 크게 하여도 학습이 잘 진행됨

→ 학습 속도 향상

과적합(Overfitting) 문제 감소

→ Dropout 같은 기법을 사용하지 않아도 됨

계수의 초기화에 덜 민감함

Sigmoid나 Tanh을 활성화 함수로 사용하여도 학습이 잘 됨

→ 이전에는 Sigmoid나 tanh을 깊은 신경망에서 사용할 경우 학습이 잘 안 되는 문제가 있었음

- 합성곱 신경망(Convolutional Neural Network, CNN)은 영상처리에 주로 이용됨
- CNN은 합성곱 연산을 이용한 합성곱 층들로 구성됨
 - 학습을 통해 합성곱 연산에 사용되는 계수들의 값을 조정함
- 합성곱 층
 - 3차원 입력 데이터에 대해 여러 개의 3차원 커널로 합성곱 연산을 수행하며 출력 데이터도 3차원임
 - 입력 채널의 수가 N 이면 커널은 $K \times K \times N$ 임
 - 커널의 개수가 M 이면 출력 채널의 수도 M 임
- CNN의 마지막에는 1~3개의 완전연결층(Fully-connected layers)를 두는 것이 일반적임
 - 완전연결층: 일반적인 신경망 층
- 합성곱 층이나 완전연결층 다음에는 일반적으로 활성화 함수를 배치함
- Pooling 층에서는 각 입력 Feature Map마다 따로 연산을 수행함
 - 주로 $K \times K$ 영역에 대해 MAX 또는 AVG 연산을 하며, stride를 넣는 것이 일반적임
- 합성곱 층에 stride나 padding을 넣기도 함
- 공변량 쉬프트 현상을 해결해서 학습을 원활하게 하기 위해 배치 정규화 층을 사용함
- 클래스 분류 문제에서는 CNN의 마지막에 SoftMax 층을 보통 사용함